

“Design and Development of a Travel Package Booking System using Flask and SQLite”

A Project Report submitted in partial fulfilment of the requirements for the course
Database Systems Engineering and Distributed Backend Development (25CS1302E)
for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

By

NALLA ROHITH :2520090191

CHALLA GOUTHAM TEJA :2520090225

L JASWANTH :2520080040

Team ID: 18 | Section: 11

Under the Esteemed Guidance of

Dr. (Mrs.) Gayatri

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

Declaration

The Project Report entitled “Design and Development of a Travel Package Booking System using Flask and SQLite” is a record of bonafide work of NALLA ROHITH – 2520090191, CHALLA GOUTHAM TEJA – 2520090225, L JASWANTH – 2520080040 (Team ID 18, Section 11), submitted in partial fulfilment for the award of B.Tech in Computer Science and Engineering to K L (Deemed to be) University. The results embodied in this report have not been copied from any other department, university or institute.

NALLA ROHITH – 2520090191
CHALLA GOUTHAM TEJA – 2520090225
L JASWANTH – 2520080040

CERTIFICATE

This is to certify that the project report entitled “Design and Development of a Travel Package Booking System using Flask and SQLite” is a bonafide work done and submitted by NALLA ROHITH – 2520090191, CHALLA GOUTHAM TEJA – 2520090225, L JASWANTH – 2520080040 (Team ID 18, Section 11), in partial fulfilment of the requirements for the course Database Systems Engineering and Distributed Backend Development (25CS1302E) towards the award of the degree of BACHELOR OF TECHNOLOGY in the Department of Computer Science and Engineering, K L (Deemed to be) University, during the academic year 2026–2027.

Signature of the Guide
Dr. (Mrs.) Gayatri

Signature of the HOD

Signature of the Course Coordinator

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance of many people. We express our sincere thanks to everyone who assisted us in one way or another in completing this project.

Our greatest appreciation goes to our guide, Dr. (Mrs.) Gayatri, Department of Computer Science and Engineering, for her tremendous support, encouragement and guidance throughout this project. Her suggestions on database design, transaction handling and testing helped us shape the system into a reliable working application.

We express our gratitude to the Head of the Department of Computer Science and Engineering and to the Course Coordinator for providing us with adequate facilities, ways and means by which we were able to complete this project-based learning activity.

We thank all the members of the teaching and non-teaching staff who assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

NALLA ROHITH – 2520090191

CHALLA GOUTHAM TEJA – 2520090225

L JASWANTH – 2520080040

Abstract

Planning a holiday often means comparing many packages by hand, remembering how many seats are left, and keeping track of what has been booked or cancelled. When this is done on paper, in spreadsheets or through messaging, it leads to double bookings, wrong prices and scattered records. This project, the Travel Package Booking System, solves this problem with a web application backed by a relational database.

The system is built with the Flask framework of Python, the SQLite relational database, Jinja2 templates and Bootstrap 5 for a responsive interface. A traveller can register and log in securely, browse six ready-made tour packages, search by destination and maximum price, view package details, book a package, cancel a booking, keep a wishlist of favourite packages and view a profile that shows active bookings and the total amount spent. Passwords are never stored in plain text; they are saved as salted hashes.

The central database feature of the project is its inventory and transaction design. Every package has an inventory row that stores the number of available slots. A booking is performed inside a BEGIN IMMEDIATE transaction that reads the slots, decrements them and inserts the booking row as one atomic unit; if no slot is left the transaction is rolled back and the traveller is told that the package is sold out. Cancellation changes the booking status to CANCELLED and releases the slot in a single transaction. All SQL is parameterised, which protects the system from SQL injection.

The application was verified with 30 test cases covering registration, login, access control, search, booking, overbooking, cancellation, wishlist, profile and security. All 30 test cases passed. The project demonstrates the practical use of normalised table design, primary and foreign keys, joins, aggregate functions and ACID transactions in a real application.

Index

S.No	Chapter	Topics	Page No.
	Preliminary	Acknowledgement Abstract	4–5
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objectives 1.5 Importance of the application 1.6 Target users / audience	7
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	10
3	Technology Stack	3.1 Front-end 3.2 Back-end 3.3 Database 3.4 Version control 3.5 APIs or external services	11
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of each layer 4.3 Deployment architecture	13
5	Design	5.1 ER diagram / database schema 5.2 Data dictionary 5.3 Normalisation 5.4 Booking flow design	15
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic 6.3 Routes and request handling 6.4 Authentication and authorisation	18
7	Features	7.1 List of main features 7.2 How each feature works	22
8	Testing	8.1 Testing approach 8.2 Integration testing 8.3 Tools used for testing 8.4 Test cases and results	25

S.No	Chapter	Topics	Page No.
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Hosting	28
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations	29
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations and optimisations	31
12	Conclusion	12.1 Summary of the project 12.2 What was achieved 12.3 Skills learned	32
13	References	Books, documentation and tutorials used	33
14	Appendices	Screenshots Complete source code (app.py) Installation / setup instructions User manual	34

CHAPTER 1

INTRODUCTION

1.1 Background of the Project

Tourism is one of the fastest-growing sectors in the world and an important contributor to national economies. Earlier, travellers depended on local travel agents, printed brochures and word of mouth to plan a trip. Today, most people prefer to browse destinations, compare prices and reserve their journey online from a laptop or a mobile phone.

A travel package is a ready-made bundle that normally includes a destination, a number of days, an itinerary of activities and a fixed price. Travel companies offer only a limited number of seats or slots for every package, because hotels, vehicles and guides have limited capacity. Managing these slots correctly is the most important task of any booking system: if two travellers are allowed to book the last slot at the same time, the company faces a double booking and an unhappy customer.

This project, the Travel Package Booking System, is a database-driven web application that allows travellers to register, search for tour packages, book them, cancel them and maintain a wishlist. It is developed as a Project Based Learning (PBL) activity for the course Database Systems Engineering and Distributed Backend Development (25CS1302E) and focuses on applying database concepts such as table design, keys, joins, aggregation and transactions to a real-life problem.

From an academic point of view, the project combines several areas of computer science:

1. Web technology: server-side rendering of HTML pages with Flask and Jinja2, styled with Bootstrap.
2. Database management: a normalised relational schema with primary keys, foreign keys and unique constraints in SQLite.
3. Transaction processing: atomic booking and cancellation that keep the slot count consistent.
4. Security: password hashing, session-based login, route protection and parameterised SQL queries.
5. Software engineering: modular route design, testing and documentation.

1.2 Problem Statement

Many small and medium travel agencies still handle package bookings through phone calls, paper registers or spreadsheets. This manual approach has several problems:

- **Double booking:** the number of remaining slots is not updated instantly, so the same slot may be sold twice.
- **Scattered records:** customer details, bookings and cancellations are stored in different places, which makes it hard to know who has booked what.

- **Pricing and data errors:** prices and package details are copied by hand and may become inconsistent.
- **Slow customer service:** a traveller cannot see available packages, prices and remaining slots without contacting the agency.
- **Weak security:** personal details and passwords in shared files can be seen or changed by anyone.

There is therefore a need for a centralised system in which package data, user data, bookings and slot availability are stored in one relational database and updated together so that the data always stays correct.

1.3 Scope of the Project

In scope. The system covers the traveller side of a booking platform: registration and login, a dashboard with summary counts, package browsing with search and price filter, package detail pages, booking with slot control, a personal booking history with cancellation, a wishlist, and a profile page. The package catalogue is stored in the database and loaded with six sample packages the first time the application starts.

Out of scope. Online payment gateways, an administrator interface for adding or editing packages, e-mail or SMS notifications, travel-date selection and multi-traveller bookings are not part of the current version. They are listed as future enhancements in Chapter 11.

1.4 Objectives of the Project

1. To design a normalised relational database (users, packages, inventory, bookings and wishlist) with proper primary and foreign keys.
2. To build a web application with secure registration, login and logout using hashed passwords and sessions.
3. To allow travellers to search packages by destination and maximum price and to view full package details.
4. To implement booking and cancellation as atomic database transactions so that slots are never oversold or lost.
5. To provide wishlist and profile features that use joins and aggregate queries.
6. To test the application with a set of functional, security and concurrency-related test cases and document the results.

1.5 Importance of the Application

- **For travellers:** they can see prices, durations and remaining slots at any time and book in a few clicks without waiting for an agent.
- **For travel agencies:** all data is held centrally, slot counts are always accurate and booking history is available for reports.
- **For learning:** the project shows how ACID transactions solve the overbooking problem that occurs in real systems such as ticketing and hotel reservation.

- **For the environment:** online booking reduces the use of paper brochures and registers.

1.6 Target Users / Audience

- **Travellers / customers:** individuals who want to find and reserve a holiday package.
- **Small travel agencies and tour operators:** who need a simple low-cost booking system.
- **Students and faculty:** as a reference project for database design and transaction handling.

CHAPTER 2

SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The application is lightweight because SQLite is an embedded database and Flask has a small footprint. The following hardware is sufficient for development and for demonstration.

Component	Minimum	Recommended
Processor	Dual-core 1.6 GHz	Quad-core 2.4 GHz or higher
RAM	2 GB	4 GB or more
Storage	200 MB free space	1 GB free space
Display	1024 x 768	1366 x 768 or higher
Network	Needed once for Bootstrap and font CDN	Broadband connection
Client device	Any device with a modern browser	Laptop, tablet or smartphone

2.2 Software Requirements

Software	Details
Operating system	Windows 10/11, Ubuntu/Linux or macOS
Programming language	Python 3.8 or later
Web framework	Flask (with Werkzeug and Jinja2, installed automatically)
Database	SQLite 3 (included with Python through the sqlite3 module)
Web browser	Google Chrome, Microsoft Edge, Mozilla Firefox or Safari (latest versions)
Front-end library	Bootstrap 5.3.0 and Google Font Poppins (loaded from CDN)

2.3 Development Tools and Frameworks

- **Visual Studio Code:** source-code editor used to write and debug app.py.
- **Git and GitHub:** version control and sharing of the source code among the three team members.
- **Postman / browser:** for sending requests and checking responses during development.
- **DB Browser for SQLite:** optional tool to inspect the tables of travel.db.
- **Draw.io:** for drawing the architecture and ER diagrams.
- **pip:** Python package installer, used with the command `pip install flask`.

CHAPTER 3

TECHNOLOGY STACK

3.1 Front-End Design

The front end is made of HTML pages that are generated on the server. All page templates are stored as Jinja2 strings inside app.py. A common layout string named BASE contains the HTML head, the navigation bar, the area for flash messages and a slot where the body of each page is inserted. A helper function page() renders the body first and then places it inside BASE, so every page automatically has the same look.

- **Bootstrap 5.3.0:** provides the responsive grid, cards, forms, buttons and alerts.
- **Custom CSS:** a brand colour (#C00000), gradient navigation bar and hero section, hover animations on cards and buttons, and a fade-in animation.
- **Poppins font:** loaded from Google Fonts for a clean modern appearance.
- **Emoji icons:** every package has an emoji stored in the database which is shown as its picture, so no image files are required.
- **Jinja2 logic:** loops, conditions and filters display package lists, slot status (green or red) and formatted prices.

3.2 Back-End Design

The back end is written in Python using the Flask micro-framework. Each web address is mapped to a Python function with the @app.route decorator. Flask handles the request, the function queries the database, and the result is returned as an HTML page or a redirect.

- **Flask:** routing, request and session handling, redirects and flash messages.
- **Werkzeug security:** generate_password_hash and check_password_hash for safe password storage.
- **sqlite3 module:** executes SQL statements; the row_factory is set to sqlite3.Row so columns can be read by name.
- **Decorators:** login_required protects private pages; teardown_appcontext closes the database connection after every request.
- **Flask session:** stores the logged-in user's id in a signed cookie.

3.3 Database

SQLite was selected as the database engine. It is a serverless, file-based relational database that supports SQL, primary and foreign keys, unique constraints and ACID transactions. The whole database is a single file named travel.db which is created automatically by the init_db() function when the application starts for the first time.

Reason for choosing SQLite	Explanation
Zero configuration	No separate database server needs to be installed or started.
Full SQL support	Joins, aggregates (COUNT, SUM, COALESCE) and sub-queries are available.
ACID transactions	BEGIN IMMEDIATE, COMMIT and ROLLBACK make booking and cancellation atomic.
Portability	The database is one file that can be copied with the project.
Suitable for PBL	Ideal for a learning project and small deployments.

3.4 Version Control

Git is used for version control and the repository is hosted on GitHub. Team members commit their work to branches, review each other's changes and merge them into the main branch. The repository link is shown as a QR code on the project poster.

3.5 APIs or External Services

The application does not call any paid or third-party web API. The only external resources are two static files loaded by the browser from a Content Delivery Network (CDN): the Bootstrap CSS file from cdn.jsdelivr.net and the Poppins font from Google Fonts. All business data comes from the local SQLite database.

CHAPTER 4

SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

The system follows a three-tier architecture. The browser is the presentation tier, the Flask application is the application (logic) tier and the SQLite file is the data tier.

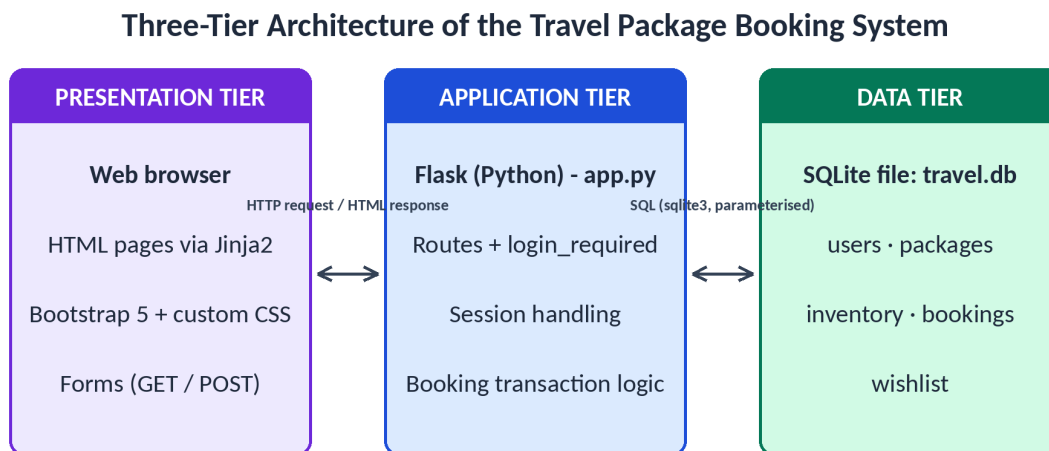


Figure 4.1: Three-tier architecture of the system

4.2 Description of Each Layer

Presentation tier (front end)

The user opens the website in a browser. The browser sends HTTP GET requests to view pages and HTTP POST requests to submit forms such as registration, login, booking, cancellation and wishlist toggling. The server answers with fully rendered HTML styled by Bootstrap.

Application tier (back end)

Flask receives every request, finds the matching route and runs the Python function. Private routes first pass through the login_required decorator, which reads the user id from the session and redirects to the login page if the user is not logged in. The function then runs SQL statements, applies business rules such as checking the available slots and finally renders a template or redirects.

Data tier (database)

The sqlite3 module opens travel.db for every request through get_db(). The database stores five tables: users, packages, inventory, bookings and wishlist. All queries use ? placeholders so user input is never joined directly into SQL text.

4.3 Deployment Architecture

For the project demonstration the application runs locally with the command `python app.py`, which starts the Flask development server on `http://127.0.0.1:5000`. The browser and server run on the same computer and the database file is created in the project folder. For a production deployment the application can be placed behind a WSGI server such as Gunicorn (Linux) or Waitress (Windows) with a reverse proxy such as Nginx. This is explained in Chapter 9.

CHAPTER 5

DESIGN

5.1 ER Diagram / Database Schema

The database consists of five tables. A user can make many bookings and can save many packages in a wishlist. A package can appear in many bookings and many wishlists, and has exactly one inventory row that stores its remaining slots. The wishlist table is the associative (junction) table that resolves the many-to-many relationship between users and packages.

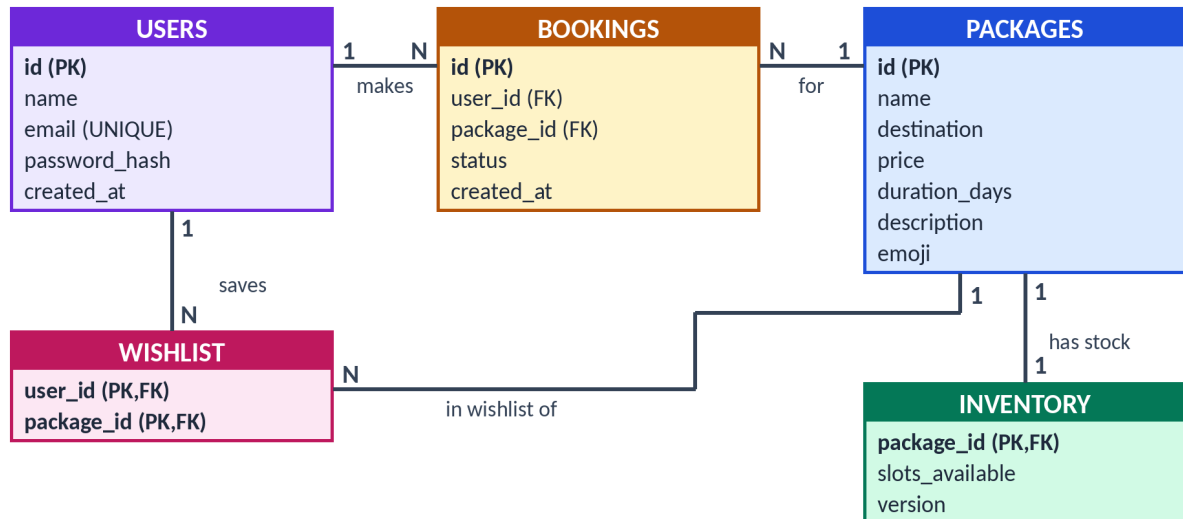


Figure 5.1: Entity-Relationship diagram

5.2 Data Dictionary

Table: users

Column	Type	Constraint / Description
id	INTEGER	Primary key, auto-increment
name	TEXT	NOT NULL - full name of the traveller
email	TEXT	UNIQUE, NOT NULL - login id, stored in lower case
password_hash	TEXT	NOT NULL - salted hash, never the plain password
created_at	TEXT	NOT NULL - registration time (ISO format)

Table: packages

Column	Type	Constraint / Description
id	INTEGER	Primary key, auto-increment
name	TEXT	NOT NULL - package name
destination	TEXT	NOT NULL - place to visit
price	REAL	NOT NULL - price in rupees
duration_days	INTEGER	NOT NULL - number of days
description	TEXT	Short description of the trip
emoji	TEXT	Icon shown for the package (default beach emoji)

Table: inventory

Column	Type	Constraint / Description
package_id	INTEGER	Primary key and foreign key to packages(id) - one row per package
slots_available	INTEGER	NOT NULL - slots still free
version	INTEGER	NOT NULL, default 0 - incremented on every booking

Table: bookings

Column	Type	Constraint / Description
id	INTEGER	Primary key, auto-increment
user_id	INTEGER	Foreign key to users(id)
package_id	INTEGER	Foreign key to packages(id)
status	TEXT	NOT NULL - CONFIRMED or CANCELLED
created_at	TEXT	NOT NULL - booking time

Table: wishlist

Column	Type	Constraint / Description
user_id	INTEGER	Part of composite primary key; foreign key to users(id)
package_id	INTEGER	Part of composite primary key; foreign key to packages(id)

5.3 Normalisation

The schema is in Third Normal Form (3NF):

- **1NF:** every column holds a single atomic value and every table has a primary key.
- **2NF:** in the wishlist table, which has a composite key, there are no other columns, so no partial dependency exists. In all other tables the key is a single column.

- **3NF:** non-key attributes depend only on the key. Slot information is kept in inventory and not repeated in packages or bookings, and the price is stored once in packages and read through a join, so there is no transitive dependency or duplicated data.

Keeping the slot counter in a separate inventory table also separates descriptive data (which rarely changes) from transactional data (which changes with every booking), and makes locking and updating the counter simple.

5.4 Booking Flow Design

The most critical design is the booking transaction. It is shown in Figure 5.2. The transaction starts with **BEGIN IMMEDIATE**, which acquires the write lock on the database so that no other booking can read or change the slot count until this transaction finishes. The slot count is checked, and only if at least one slot is available are the inventory update and the booking insert executed together and committed.

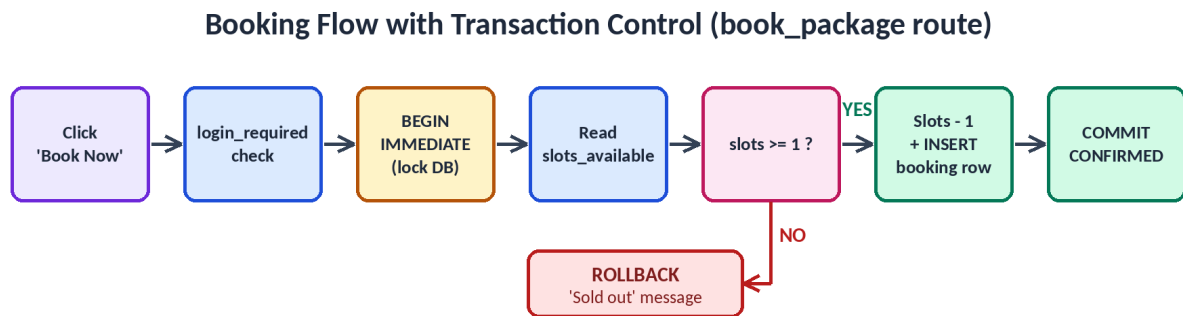


Figure 5.2: Booking flow with transaction control

CHAPTER 6

IMPLEMENTATION

Section 6.1 – Module-Wise Implementation

The application is implemented in a single Python file, `app.py`, divided into clearly marked sections. Each section is a module with its own routes.

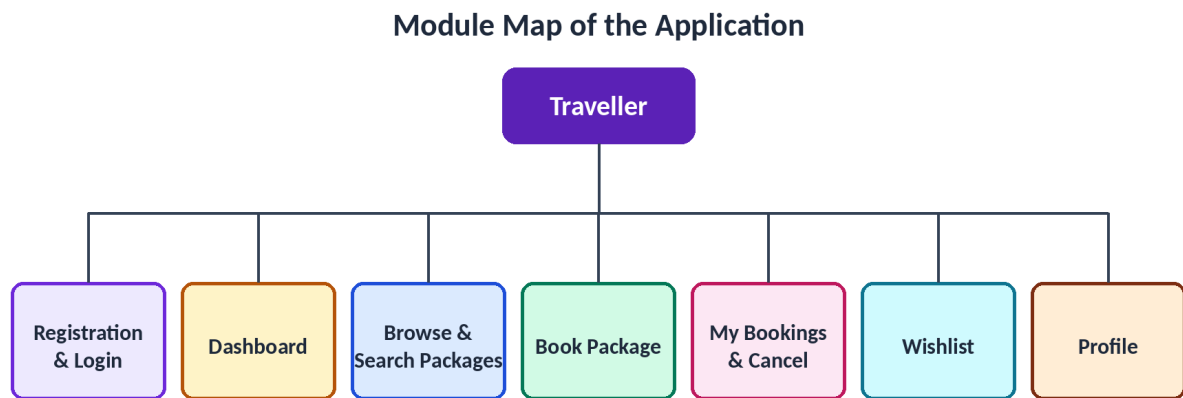


Figure 6.1: Module map of the application

Module	Routes	Implementation summary
Database setup	<code>init_db()</code> , <code>get_db()</code>	Creates the five tables if they do not exist and inserts six sample packages with 5 slots each when the packages table is empty.
Public landing	<code>/</code>	Shows a hero banner and three popular packages (a JOIN of packages and inventory). Logged-in users are redirected to the dashboard.
Authentication	<code>/register</code> , <code>/login</code> , <code>/logout</code>	Registers users with hashed passwords, checks credentials, creates and clears the session.
Dashboard	<code>/dashboard</code>	Shows three counts: packages available, the user's active bookings and wishlist items, and four shortcut buttons.
Package browsing	<code>/packages</code> , <code>/package/<id></code>	Search by destination (LIKE) and maximum price, with slots left; detail page with Book Now or Sold Out.
Booking	<code>/book/<id></code> (POST)	Atomic transaction that decrements the slot and inserts a CONFIRMED booking.
My Bookings	<code>/my-bookings</code> , <code>/cancel/<id></code>	Lists the user's bookings newest first; cancellation sets CANCELLED and returns the slot.
Wishlist	<code>/wishlist</code> , <code>/wishlist/toggle/<id></code>	Adds or removes a package with one button (toggle) and lists saved packages.
Profile	<code>/profile</code>	Shows name, email, member-since date, active bookings and total amount spent (SUM with JOIN).

The database is created by the following statements in `init_db()`:

```

CREATE TABLE IF NOT EXISTS users(

    id INTEGER PRIMARY KEY AUTOINCREMENT,

    name TEXT NOT NULL,

    email TEXT UNIQUE NOT NULL,

    password_hash TEXT NOT NULL,

    created_at TEXT NOT NULL

);

CREATE TABLE IF NOT EXISTS packages(

    id INTEGER PRIMARY KEY AUTOINCREMENT,

    name TEXT NOT NULL, destination TEXT NOT NULL,

    price REAL NOT NULL, duration_days INTEGER NOT NULL,

    description TEXT, emoji TEXT DEFAULT '🌈'

);

CREATE TABLE IF NOT EXISTS inventory(

    package_id INTEGER PRIMARY KEY REFERENCES packages(id),

    slots_available INTEGER NOT NULL,

    version INTEGER NOT NULL DEFAULT 0

);

CREATE TABLE IF NOT EXISTS bookings(

    id INTEGER PRIMARY KEY AUTOINCREMENT,

    user_id INTEGER REFERENCES users(id),

    package_id INTEGER REFERENCES packages(id),

    status TEXT NOT NULL, created_at TEXT NOT NULL

);

CREATE TABLE IF NOT EXISTS wishlist(

    user_id INTEGER REFERENCES users(id),

    package_id INTEGER REFERENCES packages(id),

    PRIMARY KEY (user_id, package_id)

);

```

6.2 Front-End Logic (UI Rendering and State Management)

The user interface is rendered on the server, so no client-side framework is needed. The `page()` helper first renders the body template with the data passed from the route and then wraps it in the BASE layout together with the current user. The navigation bar changes automatically: logged-in users see Dashboard, Packages, My Bookings, Wishlist, Profile and Logout, while visitors see Login and Register buttons.

State is handled in two ways. Short messages such as “Booking confirmed” or “Invalid email or password” are stored with `flash()` and shown once as Bootstrap alerts on the next page. The login state is kept in the Flask session cookie, which stores only the user id. All other state, such as bookings and slots, is stored in the database. The wishlist heart button in the package grid changes colour (filled or outlined) according to whether the package is in the user's wishlist.

6.3 Routes and Request Handling

The application is server-rendered, so its “APIs” are the HTTP routes described below. State-changing actions use POST requests so that they cannot be triggered by simply opening a link.

Method	URL	Login needed	Action
GET	/	No	Landing page
GET/ POST	/register	No	Show form / create account
GET/ POST	/login	No	Show form / verify credentials
GET	/logout	No	Clear session
GET	/dashboard	Yes	Summary counts and shortcuts
GET	/packages? destination=&max_price=	Yes	Search and list packages
GET	/package/<id>	Yes	Package details (404 if missing)
POST	/book/<id>	Yes	Book a package
GET	/my-bookings	Yes	Booking history
POST	/cancel/<id>	Yes	Cancel own booking
GET	/wishlist	Yes	Saved packages
POST	/wishlist/toggle/<id>	Yes	Add or remove wishlist item
GET	/profile	Yes	Profile and totals

Package search builds the SQL text dynamically but passes every value through ? placeholders:

```
query = """SELECT p.*, i.slots_available FROM packages p

            JOIN inventory i ON i.package_id=p.id WHERE 1=1"""

params = []

if dest:

    query += " AND p.destination LIKE ?"; params.append(f"%{dest}%")

if max_price:

    query += " AND p.price <= ?"; params.append(max_price)

pkgs = db.execute(query, params).fetchall()
```

6.4 Authentication and Authorisation

- **Registration:** the name and email are trimmed, the email is converted to lower case, and the password is hashed with `generate_password_hash`. The UNIQUE constraint on email raises an `IntegrityError` for duplicates, which is shown to the user as “Email already registered”.
- **Login:** the user is fetched by email and `check_password_hash` compares the typed password with the stored hash. The same message is shown for a wrong email or a wrong password, so attackers cannot find out which emails exist.
- **Session:** after login, `session["user_id"]` is set. Flask signs the cookie with the application's secret key.
- **Authorisation:** `login_required` blocks every private page for visitors. In addition, the cancel route searches the booking with both its id and the current user's id, so one user cannot cancel another user's booking.

```
def login_required(view):
    @wraps(view)
    def wrapped(*a, **kw):
        if not current_user():
            flash("Please log in first.")
            return redirect(url_for("login"))
        return view(*a, **kw)
    return wrapped

# registration: password is hashed before it is stored
db.execute("INSERT INTO users(name,email,password_hash,created_at) VALUES (?, ?, ?, ?)",
            (name, email, generate_password_hash(password),
             datetime.utcnow().isoformat()))

# login: hash comparison, session created on success
if user and check_password_hash(user["password_hash"], password):
```



```
session["user_id"] = user["id"]
```

CHAPTER 7

FEATURES

7.1 List of Main Features

1. Secure user registration and login with hashed passwords.
2. Dashboard with live counts of packages, active bookings and wishlist items.
3. Package catalogue with emoji icon, destination, duration, description, price and remaining slots.
4. Search by destination (partial, case-insensitive match) and filter by maximum price.
5. Package detail page with Book Now button, or a disabled Sold Out button when no slot is left.
6. One-click booking protected by a database transaction.
7. My Bookings page with status badges and a Cancel button for confirmed bookings.
8. Cancellation that releases the slot immediately.
9. Wishlist with add/remove toggle.
10. Profile page showing active bookings and total money spent.
11. Responsive design that works on phones, tablets and computers.
12. Flash messages that give instant feedback for every action.

7.2 How Each Feature Works

Registration and login

User view: the visitor enters name, email and password on the Register page and then logs in. After login the dashboard greets the user by name. Technical view: a row is inserted in users with a hashed password; on login the hash is verified and the user id is stored in the session.

Browsing and searching packages

User view: the Packages page shows cards for all packages. The user can type part of a destination and/or a maximum price and press Search. Technical view: a SELECT with a JOIN between packages and inventory is executed. The LIKE operator with % wildcards gives partial matching and a price condition is added only if a value was entered.

Booking a package

User view: on the package page the user presses Book Now and is taken to My Bookings with a confirmation message. If the package is sold out, a message explains it. Technical view: the booking runs in a BEGIN IMMEDIATE transaction (see Figure 5.2). The slot count is read, decremented, the version number is incremented and a CONFIRMED booking row is inserted before COMMIT. Any exception causes ROLLBACK.

```
@app.route("/book/<int:pkg_id>", methods=["POST"])

@login_required

def book_package(pkg_id):

    db = get_db()

    user = current_user()

    try:

        db.execute("BEGIN IMMEDIATE")

        row = db.execute("SELECT slots_available FROM inventory "

                        "WHERE package_id=?", (pkg_id,)).fetchone()

        if row is None or row["slots_available"] < 1:

            db.execute("ROLLBACK")

            flash("Sorry, this package is sold out.")

            return redirect(url_for("package_detail", pkg_id=pkg_id))

        db.execute("UPDATE inventory SET slots_available = slots_available - 1, "

                  "version = version + 1 WHERE package_id=?", (pkg_id,))

        db.execute("INSERT INTO bookings(user_id, package_id, status, created_at) "

                  "VALUES (?, ?, ?, ?)",

                  (user["id"], pkg_id, "CONFIRMED", datetime.utcnow().isoformat()))

        db.execute("COMMIT")
```

```

        flash("Booking confirmed! Check 'My Bookings'.")

    except Exception as e:

        db.execute("ROLLBACK")

        flash(f"Booking failed: {e}")

    return redirect(url_for("my_bookings"))

```

Cancelling a booking

User view: a Cancel button is displayed only for confirmed bookings. After pressing it, the status badge changes to CANCELLED. **Technical view:** the booking is selected using both booking id and user id, then in one transaction the status becomes CANCELLED and the slot count of the package is increased by one. A second cancel request on the same booking does nothing, so a slot can never be released twice.

```

@app.route("/cancel/<int:booking_id>", methods=["POST"])

@login_required

def cancel_booking(booking_id):

    db = get_db()

    user = current_user()

    b = db.execute("SELECT * FROM bookings WHERE id=? AND user_id=?",

                    (booking_id, user["id"])).fetchone()

    if b and b["status"] == "CONFIRMED":

        db.execute("BEGIN IMMEDIATE")

        db.execute("UPDATE bookings SET status='CANCELLED' WHERE id=?", (booking_id,))

        db.execute("UPDATE inventory SET slots_available = slots_available + 1 "

                    "WHERE package_id=?", (b["package_id"],))

        db.execute("COMMIT")

        flash("Booking cancelled and slot released.")

    return redirect(url_for("my_bookings"))

```

Wishlist

User view: the heart button on a package saves it; pressing it again removes it. **Technical view:** the route checks whether a (user_id, package_id) row exists; if yes it is deleted, otherwise inserted. The composite primary key prevents duplicates.

Profile

User view: the profile shows the user's details, the number of active bookings and the total spent. Technical view: the total is calculated with a SUM over confirmed bookings joined with packages; COALESCE returns 0 when there are no bookings. Cancelled bookings are excluded.

```
SELECT COALESCE(SUM(p.price),0) s
FROM bookings b JOIN packages p ON p.id = b.package_id
WHERE b.user_id = ? AND b.status = 'CONFIRMED'
```

CHAPTER 8

TESTING

8.1 Postman Testing (Front-End / Back-End)

Because the application returns HTML pages and redirects instead of JSON, each route was tested by sending real HTTP requests and checking the status code, redirect location, page content and the resulting rows in the database. The same requests can be sent from Postman or the browser: GET requests for pages, and POST requests with form fields (name, email, password) for registration, login, booking, cancellation and wishlist toggling. Postman should keep cookies enabled so that the session created at login is reused for private routes.

The 30 test cases listed in Section 8.4 were executed automatically using the Flask test client against a freshly created database. After each action, the database was queried directly to confirm that the correct rows had been inserted, updated or left unchanged.

8.2 Integration Testing

Integration tests checked that the layers work together from the browser request down to the database and back:

- Register → Login → Dashboard: the new user can log in and the session grants access to private pages.
- Packages → Detail → Book → My Bookings: a booking changes the inventory (slots decreased, version increased) and appears in the booking list.
- Cancel → Inventory: cancelling restores the slot, and cancelling twice does not release a second slot.
- Overbooking: six consecutive booking attempts on a package with five slots produced exactly five confirmed bookings and one rejection; slots never became negative.
- Booking → Profile: the total spent equals the sum of the prices of confirmed bookings ($5 \times 15999 = 79995$ in the test).
- Logout → private pages: after logout, every private route redirects to the login page.

8.3 Tools Used for Testing

Tool	Purpose
Flask test client (Python)	Sends GET/POST requests to the routes without starting a server; keeps cookies between requests.
sqlite3 module	Reads the database directly after each action to verify the stored data.
Postman	Manual verification of routes, form POST requests and redirects.
Web browser (Chrome / Edge)	Manual user-interface testing and responsive-design checks.
DB Browser for SQLite	Visual inspection of table contents.

8.4 Test Cases and Results

A total of 30 test cases were executed. 30 passed and 0 failed, a pass rate of 100%.

ID	Module	Test description	Expected result	Actual result	Status
TC01	Landing	Open home page without login	Page loads (200) with popular packages	HTTP 200, packages shown	Pass
TC02	Access control	Open dashboard without login	Redirect to login page	HTTP 302 -> /login	Pass
TC03	Registration	Register new user	User saved; redirect to login	HTTP 302; users=1	Pass
TC04	Security	Check password storage	Password stored as hash, not plain text	Stored value starts with script:32768:8...	Pass
TC05	Registration	Email stored in lower case	email = rohith@test.com	Stored: rohith@test.com	Pass
TC06	Registration	Register with duplicate email	Rejected with 'Email already registered'	Message shown; users still 1	Pass
TC07	Login	Login with wrong password	'Invalid email or password' shown	Message shown	Pass
TC08	Login	Login with unknown email	'Invalid email or password' shown	Message shown	Pass
TC09	Login	Login with valid credentials	Dashboard opens with welcome message	Dashboard displayed	Pass
TC10	Packages	View all packages	6 packages listed	6 packages listed	Pass
TC11	Search	Search destination 'goa' (case-insensitive)	Only Goa package shown	1 result: Goa Beach Escape	Pass
TC12	Search	Filter max price 15000	2 packages (Goa 12999, Manali 10999)	2 packages shown (Goa 12999, Manali 10999)	Pass
TC13	Search	Search non-existing destination	No packages displayed	0 results	Pass
TC14	Package detail	Open package 1	Details and Book Now button	Shows 5 slots, Book Now	Pass
TC15	Package detail	Open invalid package id	404 Package not found	HTTP 404	Pass
TC16	Booking	Book package 1	Booking CONFIRMED, slots 5 -> 4, version 0 -> 1	slots=4, version=1	Pass
TC17	My Bookings	View bookings	Booking with CONFIRMED badge and Cancel button	Listed with Cancel button	Pass
TC18	Cancellation	Cancel the booking	Status CANCELLED, slot released (4 -> 5)	slots=5	Pass
TC19	Cancellation	Cancel same booking again	No change; slot not released twice	slots stay 5	Pass
TC20	Overbooking	Book package 2 six times (only 5 slots)	5 confirmed; 6th rejected as sold out; slots never negative	5 confirmed, 6th rejected, slots=0	Pass
TC21	Overbooking	Sold-out package page	Book button replaced by disabled 'Sold Out'	Sold Out shown	Pass
TC22	Booking	Book non-existing package	Rejected, no booking created	No row created	Pass
TC23	Wishlist	Add package 3 to wishlist	Row inserted; shown on wishlist page	Manali listed	Pass

ID	Module	Test description	Expected result	Actual result	Status
TC24	Wishlist	Toggle again	Row removed	Wishlist empty	Pass
TC25	Profile	Open profile	Name, email, active bookings and total spent	Profile details shown	Pass
TC26	Profile	Total spent calculation	Equals sum of confirmed package prices (5 x 15999)	Total = 79995	Pass
TC27	Authorization	Another user tries to cancel someone else's booking	Request ignored; booking stays CONFIRMED	Booking unchanged	Pass
TC28	Logout	Logout then open profile	Session cleared; redirected to login	HTTP 302 -> / login	Pass
TC29	Access control	Open packages after logout	Redirect to login	HTTP 302	Pass
TC30	Security	SQL injection in destination search	Parameterised query: no data leak/no error	HTTP 200, 0 results	Pass

Observation: all functional, access-control and security tests passed. The overbooking test (TC20) confirms that the transaction design prevents a package from being sold beyond its capacity, and TC30 confirms that a typical SQL-injection string in the search box returns no data and causes no error because the query is parameterised.

CHAPTER 9

DEPLOYMENT

9.1 Steps to Deploy

Local deployment (used for the demonstration)

1. Install Python 3.8 or later from python.org and verify with `python --version`.
2. Copy the project folder containing `app.py` to the computer.
3. Open a terminal in the folder and run: `pip install flask`
4. Start the application with: `python app.py` (on Windows you can also use `py app.py`).
5. Open `http://127.0.0.1:5000` in a web browser.
6. On the first start, the file `travel.db` is created automatically and six sample packages are inserted.

Optional: virtual environment

```
python -m venv venv
```

```
venv\Scripts\activate (Linux/macOS: source venv/bin/activate)
```

```
pip install flask
```

```
python app.py
```

9.2 Environment Configuration

Setting	Current value / recommendation
Database file	DB = "travel.db" in the project folder. Delete the file to reset all data.
Secret key	app.secret_key is set to a development value. For real use, replace it with a long random value, preferably read from an environment variable.
Debug mode	app.run(debug=True) enables auto-reload and the debugger for development. It must be turned off in production.
Port	Flask default 5000; change with app.run(port=...).
Dependencies	Only Flask is required (Werkzeug and Jinja2 come with it).

9.3 Hosting of Front-End and Backend

Since the front end is produced by the same Flask application, there is a single deployable unit. For internet hosting, the application can be run with a production WSGI server (for example gunicorn app:app on Linux or waitress-serve on Windows) behind Nginx, or on a free-tier Python host such as PythonAnywhere or Render. The SQLite file must be stored on persistent storage. If many users are expected, the database can be changed to MySQL or PostgreSQL with only the connection and SQL dialect details adjusted, because the schema is already normalised.

CHAPTER 10

CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

- **Overbooking:** when the slot count was first handled with separate read and update statements, two quick requests could both see the last slot as free.
- **Releasing slots on cancellation:** a repeated cancel request could add the slot back twice.
- **Page template management:** keeping many HTML templates inside one Python file made quoting and nesting of Jinja2 strings confusing.
- **Password security:** storing passwords safely and giving a login error that does not reveal whether the email exists.
- **Search filter:** building an SQL query with optional conditions without risking SQL injection.
- **Team coordination:** three members changing the same file led to merge conflicts in Git.

10.2 Solutions Applied

Issue	Solution
Overbooking	Booking runs in a BEGIN IMMEDIATE transaction; slots are checked and updated before COMMIT, with ROLLBACK on failure.
Double release of slots	Cancellation is allowed only when status is CONFIRMED and is done inside one transaction.
Template handling	A shared BASE layout and a page() helper reuse the layout; the code is separated by comment banners.
Password security	generate_password_hash / check_password_hash and one common error message.
Dynamic search	The query text is extended only with fixed fragments and all values are passed as ? parameters.
Merge conflicts	Each member worked on separate sections and used short-lived branches.

10.3 Current Limitations or Known Bugs

- **No payment:** booking is confirmed immediately without any payment step.
- **No administrator interface:** packages can be changed only directly in the database.
- **No travel date or traveller count:** a booking reserves one slot and is not tied to a date.
- **No CSRF tokens:** POST forms are not protected against cross-site request forgery.
- **Fixed secret key and debug mode:** app.secret_key is hard-coded and debug=True is used; both must be changed before public hosting.

- **Weak input validation:** there is no password-length rule, and max_price is not checked to be numeric (an invalid value simply returns no result).
- **Cancel route has no error handling:** unlike the booking route it does not ROLLBACK on an unexpected exception.
- **Unused version column:** the version field is incremented on booking but is not yet used for optimistic-lock checks.
- **SQLite write concurrency:** SQLite allows one writer at a time, which is fine for a demonstration but limits very high traffic.
- **Deprecated call:** datetime.utcnow() is deprecated in newer Python versions.

CHAPTER 11

FUTURE ENHANCEMENTS

11.1 Planned Features

- Admin panel to add, edit and delete packages and to manage slots.
- Payment gateway integration (UPI, cards, net banking) with a payments table and invoices.
- Travel-date selection and number of travellers per booking, with date-wise inventory.
- E-mail / SMS confirmation and cancellation notices.
- Reviews and ratings for each package.
- Refund rules on cancellation based on how close the trip is.
- Package images, categories and sorting (cheapest first, duration, popularity).

11.2 Possible Integrations or Optimisations

- Move to MySQL or PostgreSQL and a connection pool for heavy traffic.
- Split app.py into modules (blueprints) and move templates to separate HTML files.
- Provide a REST API with JSON so that a React or mobile front end can be added.
- Add CSRF protection (Flask-WTF), rate limiting on login and password rules.
- Store the secret key and database path in environment variables and disable debug mode.
- Use the version column for optimistic locking and add indexes on user_id and package_id.
- Add automated unit tests to a CI pipeline (GitHub Actions).

CHAPTER 12

CONCLUSION

12.1 Summary of the Project

The Travel Package Booking System is a complete database-driven web application that lets travellers register, search, book and cancel tour packages, maintain a wishlist and view their profile. It uses Flask for the application logic, SQLite for storage and Bootstrap for a responsive interface. The design centres on a normalised five-table schema and on transactions that keep the slot inventory correct.

12.2 What Was Achieved

- A normalised (3NF) relational schema with primary keys, foreign keys, a unique constraint and a junction table.
- Secure authentication with hashed passwords, sessions, protected routes and ownership checks.
- Atomic booking and cancellation that prevent overbooking and double release of slots.
- Search and filter, wishlist and profile features that use joins and aggregate queries.
- Parameterised SQL throughout, which resisted the SQL-injection test.
- Thirty test cases executed with 100% pass rate.
- Complete documentation with diagrams, a data dictionary, code listings and setup instructions.

All the objectives listed in Section 1.4 were met.

12.3 Skills Learned During Development

- **Database skills:** ER modelling, normalisation, SQL joins, aggregates and ACID transactions.
- **Backend skills:** Flask routing, sessions, decorators, templates and flash messages.
- **Security awareness:** password hashing, access control and SQL-injection prevention.
- **Testing:** designing test cases, checking the database state and recording results.
- **Teamwork:** using Git and GitHub, dividing work and writing project documentation.

CHAPTER 13

REFERENCES

Books

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson Education.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7th ed.). McGraw-Hill Education.

Pressman, R. S., & Maxim, B. R. (2019). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.

Grinberg, M. (2018). *Flask Web Development* (2nd ed.). O'Reilly Media.

Documentation and tutorials

Pallets Projects. *Flask Documentation*. <https://flask.palletsprojects.com>

Pallets Projects. *Jinja Template Designer Documentation*. <https://jinja.palletsprojects.com>

Pallets Projects. *Werkzeug Utilities – Password Hashing*. <https://werkzeug.palletsprojects.com>

SQLite Consortium. *SQLite Documentation – Transactions and BEGIN IMMEDIATE*. <https://www.sqlite.org/docs.html>

Python Software Foundation. *sqlite3 – DB-API 2.0 interface for SQLite databases*. <https://docs.python.org/3/library/sqlite3.html>

Bootstrap Team. *Bootstrap 5.3 Documentation*. <https://getbootstrap.com/docs/5.3>

Google Fonts. *Poppins*. <https://fonts.google.com/specimen/Poppins>

OWASP Foundation. *SQL Injection Prevention Cheat Sheet*. <https://owasp.org>

W3Schools / GeeksforGeeks / MDN Web Docs – *HTML, CSS, Python and SQL tutorials*.

CHAPTER 14

APPENDICES

A. Screenshots of the App

The following screenshots were captured from the running application (python app.py, <http://127.0.0.1:5000>) using a test account with sample bookings, so that every page shows real data from the database.

Screenshot 1: Landing page for visitors

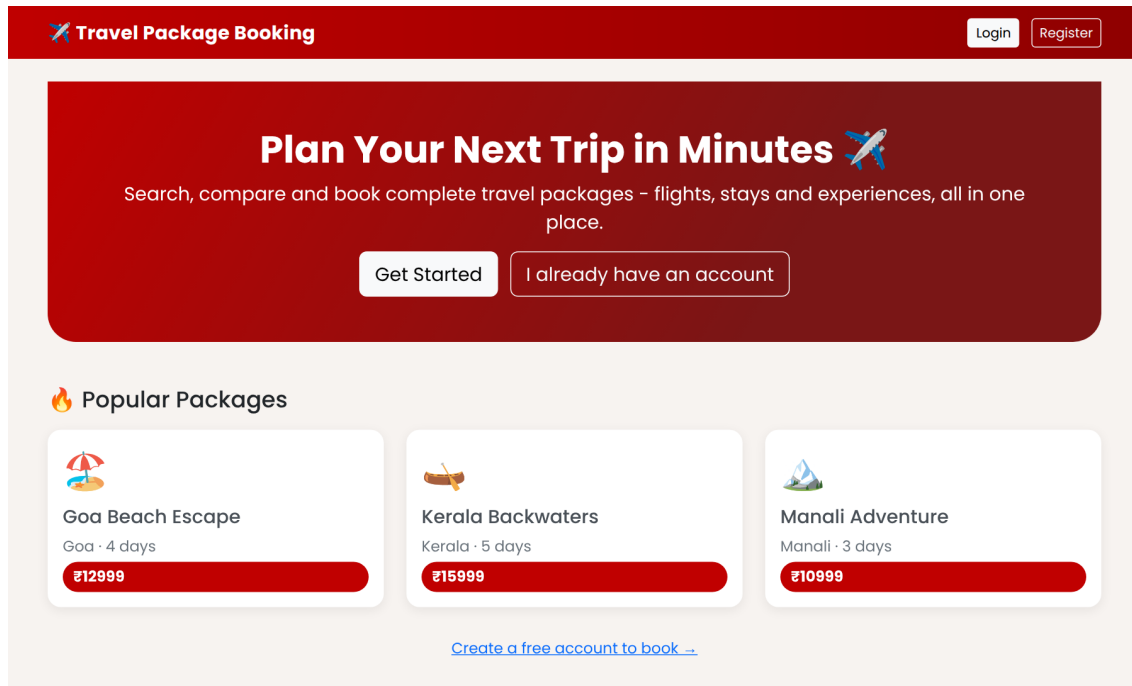


Figure A.1: Landing page for visitors

Screenshot 2: Register page

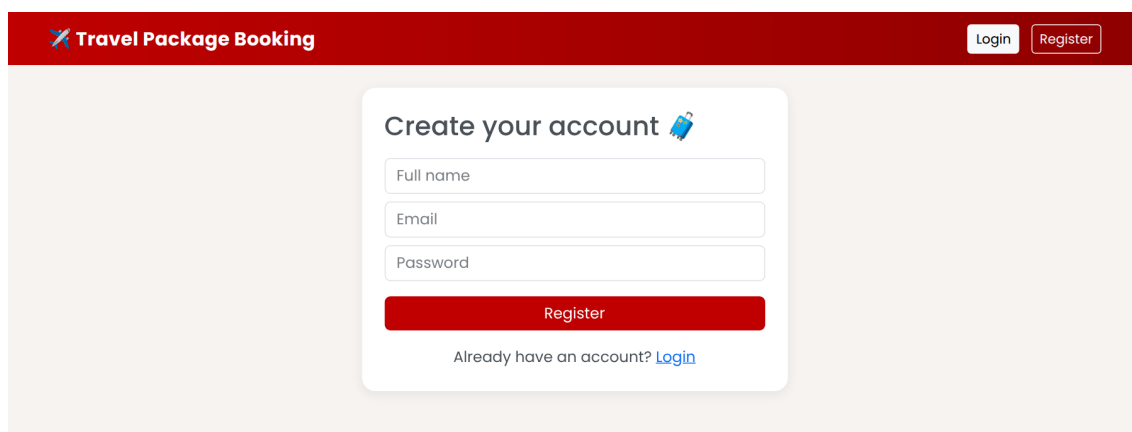


Figure A.2: Register page

Screenshot 3: Login page

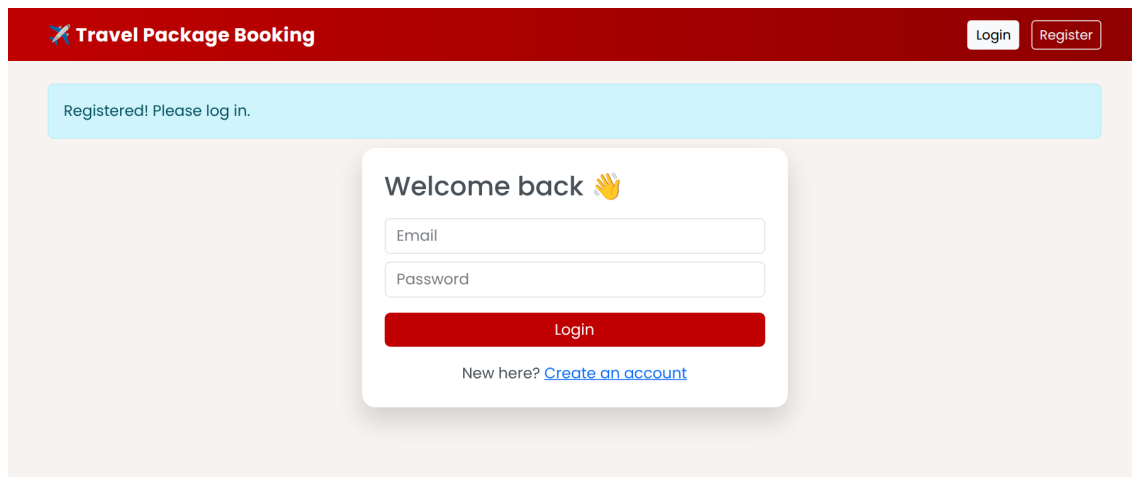


Figure A.3: Login page

Screenshot 4: Dashboard with package, booking and wishlist counts

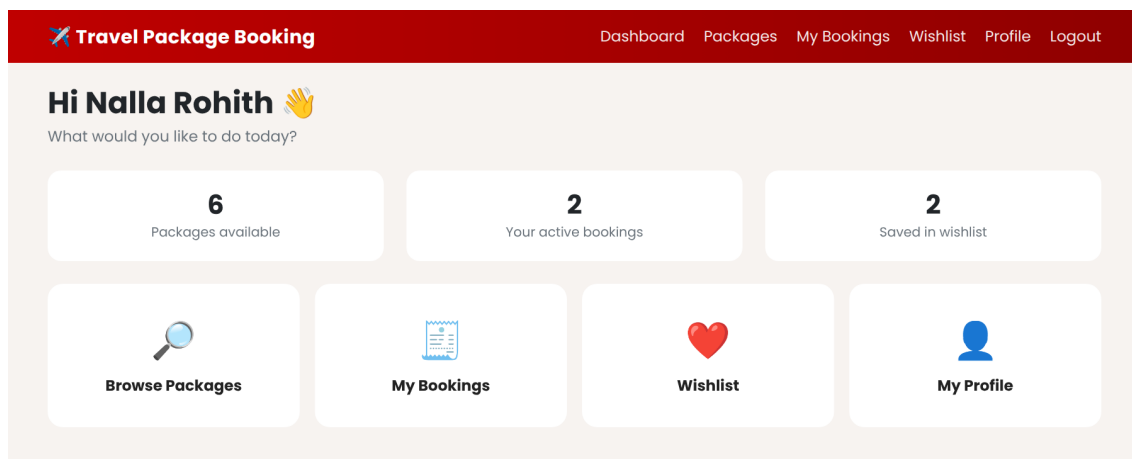


Figure A.4: Dashboard with package, booking and wishlist counts

Screenshot 5: Browse Packages page

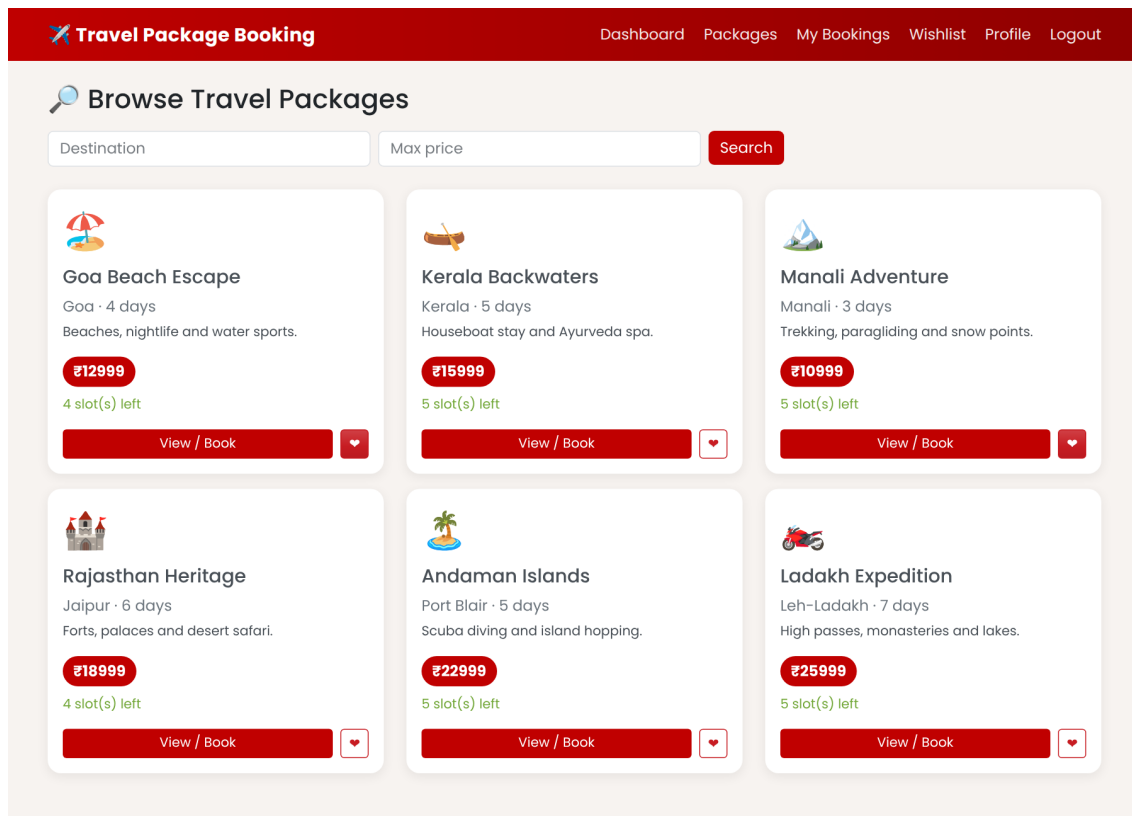


Figure A.5: Browse Packages page

Screenshot 6: Search by destination (“goa”)

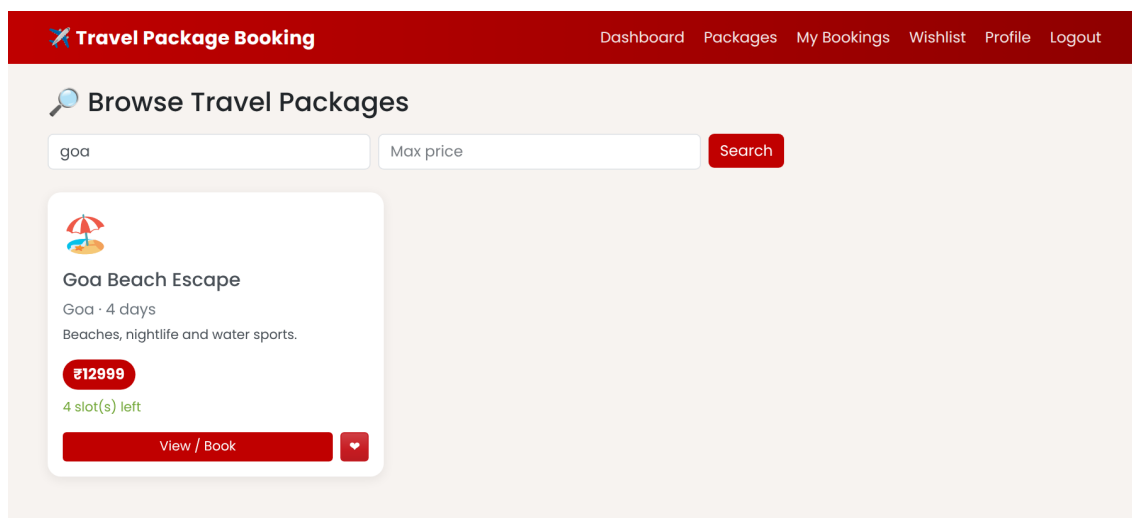


Figure A.6: Search by destination (“goa”)

Screenshot 7: Package detail page with Book Now

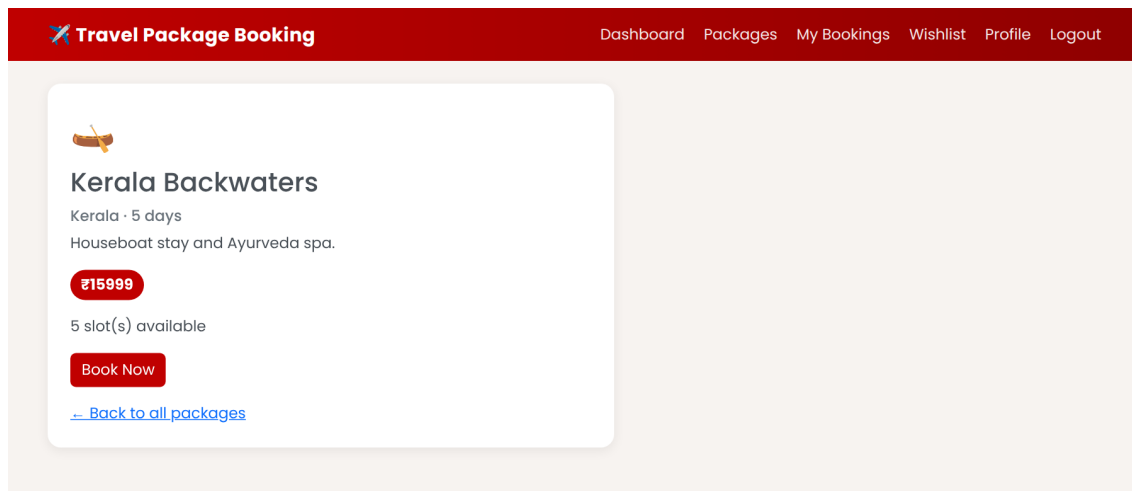


Figure A.7: Package detail page with Book Now

Screenshot 8: My Bookings (confirmed and cancelled)

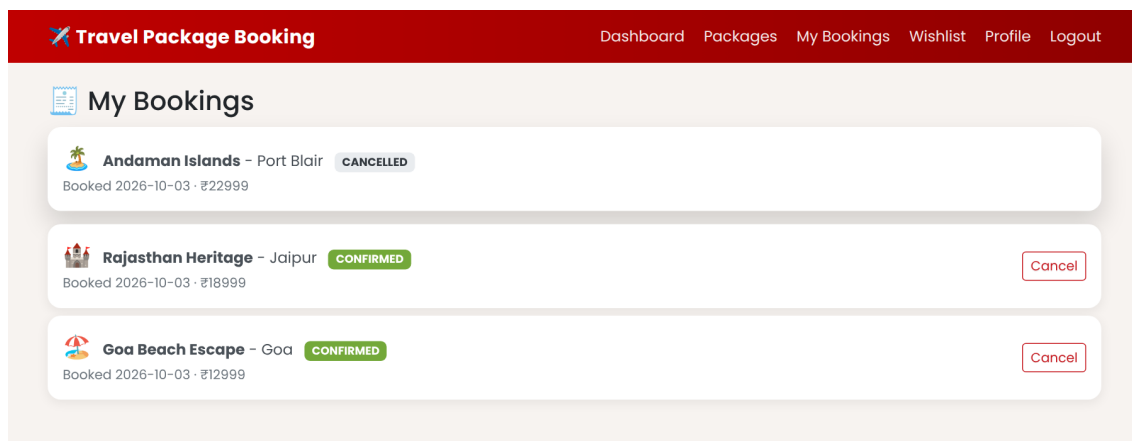


Figure A.8: My Bookings (confirmed and cancelled)

Screenshot 9: Wishlist

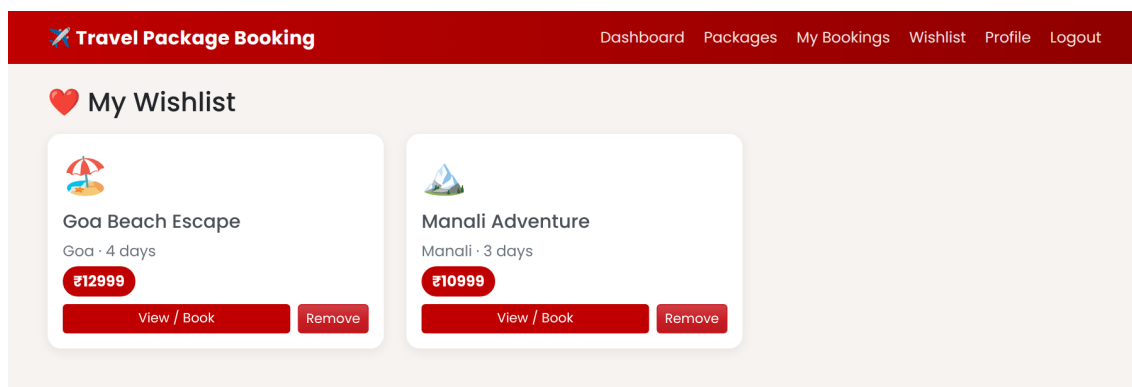


Figure A.9: Wishlist

Screenshot 10: Profile with active bookings and total spent

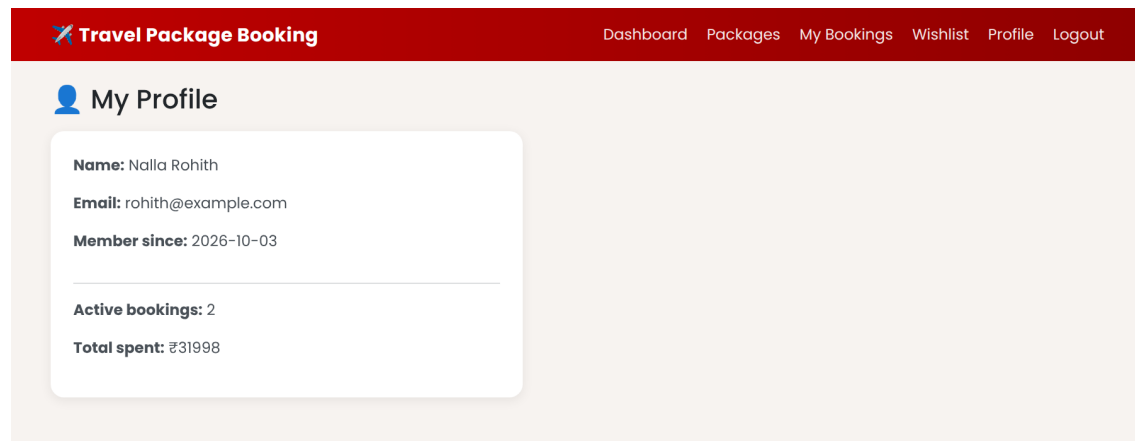


Figure A.10: Profile with active bookings and total spent

B. Sample Code Snippets

The key snippets (database schema, search query, authentication, booking, cancellation and profile query) are explained in Chapters 6 and 7. The complete source code of the application, app.py, is given below.

3 **Run:**

4 `pip install flask`

5 `python app.py` (or: `py app.py`)

6 Then open `http://127.0.0.1:5000`


```
9 import sqlite3
```

```
10 from datetime import datetime
```

```
11 from functools import wraps
```

```
12 from flask import Flask, g, request, redirect, url_for, session, render_template_string, flash
```

```
13 from werkzeug.security import generate_password_hash, check_password_hash
```



```
15 app = Flask(__name__)
```



```
16 app.secret_key = "dev-secret-key-change-me"
```

```
17 DB = "travel.db"
```



```
20 def get_db():
```

```
21         if "db" not in g:
```

```
g.db = sqlite3.connect(DB)
```

```
g.db.row_factory = sqlite3.Row
```


24 `return g.db`

26 @app.teardown_appcontext

```
27 def close_db(exc):
```

```
28         db = g.pop("db", None)
```

```
29         if db is not None:
```



```
32 def init_db():
```

```
33      conn = sqlite3.connect(DB)
```

34 `conn.executescript("""`

35 **CREATE TABLE IF NOT EXISTS users (**


```
email TEXT UNIQUE NOT NULL,
```

```
password_hash TEXT NOT NULL,
```



```
id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
name TEXT NOT NULL,
```

`destination TEXT NOT NULL,`


```
duration_days INTEGER NOT NULL,
```



```
51      CREATE TABLE IF NOT EXISTS inventory(
```

```
package_id INTEGER PRIMARY KEY REFERENCES packages(id),
```

```
slots_available INTEGER NOT NULL,
```



```
CREATE TABLE IF NOT EXISTS bookings(
```



```
user_id INTEGER REFERENCES users(id),
```

```
package_id INTEGER REFERENCES packages(id) ,
```


63 `CREATE TABLE IF NOT EXISTS wishlist(`


```
user_id INTEGER REFERENCES users(id),
```

```
package_id INTEGER REFERENCES packages(id) ,
```

PRIMARY KEY (user_id, package_id)


```
69         if conn.execute("SELECT COUNT(*) FROM packages").fetchone()[0] == 0:
```

70

```
sample = [
```

71

"🌈",

("Goa Beach Escape", "Goa", 12999, 4, "Beaches, nightlife and water sports.",


```
("Kerala Backwaters", "Kerala", 15999, 5, "Houseboat stay and Ayurveda spa.",  
"🌴"),
```

```
("Manali Adventure", "Manali", 10999, 3, "Trekking, paragliding and snow points.",  
"🏔️"),
```

("Rajasthan Heritage", "Jaipur", 18999, 6, "Forts, palaces and desert safari.",
"13"),

75

"🌴",

("Andaman Islands", "Port Blair", 22999, 5, "Scuba diving and island hopping.",

```
("Ladakh Expedition", "Leh-Ladakh", 25999, 7, "High passes, monasteries and  
lakes.", "🏔️"),
```



```
for name, dest, price, dur, desc, emoji in sample:
```

```
cur = conn.execute(
```



```
80          "INSERT INTO packages(name,destination,price,duration_days,description,emoji)
VALUES (?, ?, ?, ?, ?, ?) ",
```

```
(name, dest, price, dur, desc, emoji)
```

```
82 conn.execute("INSERT INTO inventory(package_id, slots_available, version) VALUES  
(?, ?, 0)",
```


85 `conn.close()`


```
88 def current_user():
```

```
89         uid = session.get("user_id")
```

90 **if not uid:**


```
92         return get_db().execute("SELECT * FROM users WHERE id=?", (uid,)).fetchone()
```



```
94 def login_required(view):
```



```
96     def wrapped(*a, **kw):
```

```
if not current_user():
```

```
flash("Please log in first.")
```

```
return redirect(url_for("login"))
```

```
100         return view(*a, **kw)
```

```
101         return wrapped
```



```
104 BASE = ""
```

```
105  <!doctype html><html><head>
```

```
106 <meta charset="utf-8"><meta name="viewport" content="width=device-width, initial-scale=1">
```

107 <title>Travel Package Booking</title>

```
108 <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/  
bootstrap.min.css">
```

109 <link rel="preconnect" href="https://fonts.googleapis.com">

```
110 <link href="https://fonts.googleapis.com/css2?family=Poppins:wght@400;600;700&display=swap"
rel="stylesheet">
```

111 <style>


```
112 :root{ --brand:#C00000; --brand-dark:#8f0000; }
```

```
113  body{ font-family:'Poppins',sans-serif; background:#f7f3f0; }
```

```
114 .navbar{ background:linear-gradient(90deg,var(--brand),var(--brand-dark)); }
```

```
115 .navbar-brand{ font-weight:700; letter-spacing:.3px; }
```

```
116 .nav-link{ color:#fff !important; opacity:.9; transition:.2s; }
```

```
117 .nav-link:hover{ opacity:1; transform:translateY(-1px); }
```

```
118 .btn-brand{ background:var(--brand); color:#fff; border:none; transition:.2s; }
```

```
119 .btn-brand:hover{ background:var(--brand-dark); color:#fff; transform:translateY(-2px); box-  
shadow:0 6px 14px rgba(192,0,0,.35); }
```



```
120     .card{ border:none; border-radius:14px; box-shadow:0 3px 12px rgba(0,0,0,.07);  
transition:.25s; }
```

```
121 .card:hover{ transform:translateY(-6px); box-shadow:0 14px 28px rgba(0,0,0,.15); }
```

```
122 .fade-in{ animation:fadeIn .5s ease-in both; }
```

```
123     @keyframes fadeIn{ from{opacity:0; transform:translateY(12px);} to{opacity:1;  
transform:translateY(0);} }
```

```
124 .hero{ background:linear-gradient(120deg,var(--brand),#7a1616 70%); color:#fff; border-  
radius:0 0 30px 30px; }
```

```
125 .feature-btn{ display:block; padding:2rem 1rem; border-radius:16px; text-decoration:none;  
color:#222;
```

```
background:#fff; text-align:center; font-weight:600; transition:.25s; }
```

```
127 .feature-btn:hover{ background:var(--brand); color:#fff; transform:translateY(-6px)  
scale(1.02); }
```



```
128 .feature-emoji{ font-size:2.4rem; display:block; margin-bottom:.4rem; }
```

```
129 .stat-box{ border-radius:14px; padding:1.2rem; background:#fff; }
```

```
130 .price-tag{ background:var(--brand); color:#fff; padding:.25rem .7rem; border-radius:20px;  
font-weight:600; }
```

131 </style>

132 </head><body>

```
133  {% if user %}
```

```
134 <nav class="navbar navbar-dark navbar-expand-lg">
```

135 <div class="container">

136

```
<a class="navbar-brand" href="{{ url_for('dashboard') }}"> Travel Package Booking</a>
```


138

```
<a class="nav-link d-inline-block me-3" href="{{ url_for('dashboard') }}">Dashboard</a>
```

139

```
<a class="nav-link d-inline-block me-3" href="{{ url_for('packages') }}">Packages</a>
```

```
140      <a class="nav-link d-inline-block me-3" href="{{ url_for('my_bookings') }}">My Bookings</a>
```

```
141      <a class="nav-link d-inline-block me-3" href="{{ url_for('wishlist') }}">Wishlist</a>
```

142

```
<a class="nav-link d-inline-block me-3" href="{{ url_for('profile') }}">Profile</a>
```

143

```
<a class="nav-link d-inline-block" href="{{ url_for('logout') }}">Logout</a>
```


145 </div>

146 </nav>

```
147  {% else %}
```

```
148 <nav class="navbar navbar-dark navbar-expand-lg">
```

149 <div class="container">

150

```
<a class="navbar-brand" href="{{ url_for('index') }}"> Travel Package Booking</a>
```



```
<a class="btn btn-sm btn-light me-2" href="{{ url_for('login') }}">Login</a>
```

```
<a class="btn btn-sm btn-outline-light" href="{{ url_for('register') }}">Register</a>
```



```
157  {% endif %}
```

```
158 <div class="container mt-4 mb-5 fade-in">
```

```
159     {% with messages = get_flashed_messages() %}
```



```
160      {% for m in messages %}<div class="alert alert-info">{{ m }}</div>{% endfor %}
```

```
161      {% endwith %}
```

```
162      {{ body|safe }}
```

163 </div>

164 </body></html>


```
167 def page(body, **ctx):
```



```
168     return render_template_string(BASE, body=render_template_string(body, **ctx),
user=current_user())
```


170 # ===== Public landing

```
171 @app.route("/")
```

```
172 def index():
```

```
173         if current_user():
```

```
174         return redirect(url_for("dashboard"))
```

```
175         db = get_db()
```



```
176 packages = db.execute("""SELECT p.*, i.slots_available FROM packages p
```

```
JOIN inventory i ON i.package_id=p.id LIMIT 3""").fetchall()
```

```
178     body = ""
```



```
<h1 class="fw-bold">Plan Your Next Trip in Minutes ✈️</h1>
```

181 <p class="fs-5">Search, compare and book complete travel packages - flights, stays and experiences, all in one place.</p>

```
<a class="btn btn-light btn-lg me-2" href="{{ url_for('register') }}">Get Started</a>
```

```
183      <a class="btn btn-outline-light btn-lg" href="{{ url_for('login') }}">I already have an  
account</a>
```


185 <h4 class="mb-3">🔥 Popular Packages</h4>


```
187      {% for p in packages %}
```



```
<span class="feature-emoji">{{ p['emoji'] }}</span>
```

```
<h5>{{ p['name'] }}</h5>
```



```
<p class="text-muted mb-1">{{ p['destination'] }} · {{ p['duration_days'] }} days</p>
```

```
<span class="price-tag">₹{{ '%.0f'|format(p['price']) }}</span>
```


196 {% endfor %}


```
198      <p class="text-center mt-3"><a href="{{ url_for('register') }}">Create a free account to  
book &rarr;</a></p>
```



```
200         return page(body, packages=packages)
```



```
203 @app.route("/register", methods=["GET", "POST"])
```

```
204 def register():
```

```
205         if request.method == "POST":
```

```
name = request.form["name"].strip()
```

```
email = request.form["email"].strip().lower()
```



```
password = request.form["password"]
```

```
db = get_db()
```



```
211 db.execute("INSERT INTO users(name,email,password_hash,created_at) VALUES  
(?,?,?,?)",
```

```
212                                     (name, email, generate_password_hash(password),  
datetime.utcnow().isoformat()))
```



```
flash("Registered! Please log in.")
```

215

```
return redirect(url_for("login"))
```



```
except sqlite3.IntegrityError:
```

```
flash("Email already registered.")
```

```
218     body = ""
```

219 <div class="row justify-content-center">


```
<h3 class="mb-3">Create your account 📦</h3>
```



```
<input class="form-control mb-2" name="name" placeholder="Full name" required>
```

225
required>

```
<input class="form-control mb-2" name="email" type="email" placeholder="Email"
```

```
226         <input class="form-control mb-3" name="password" type="password"
placeholder="Password" required>
```

```
<button class="btn btn-brand w-100">Register</button>
```



```
229         <p class="text-center mt-3 mb-0">Already have an account? <a  
href="{{ url_for('login') }}">Login</a></p>
```


234 **return page(body)**


```
236 @app.route("/login", methods=["GET", "POST"])
```

```
237 def login():
```

```
238     if request.method == "POST":
```

```
email = request.form["email"].strip().lower()
```



```
password = request.form["password"]
```

241

```
user = get_db().execute("SELECT * FROM users WHERE email=?", (email,)).fetchone()
```

242

```
if user and check_password_hash(user["password_hash"], password):
```

```
session["user_id"] = user["id"]
```

```
flash(f"Welcome back, {user['name']}!")
```

```
return redirect(url_for("dashboard"))
```

```
flash("Invalid email or password.")
```

```
247     body = ""
```


248 <div class="row justify-content-center">


```
<h3 class="mb-3">Welcome back 🙌</h3>
```


253
required>

```
<input class="form-control mb-2" name="email" type="email" placeholder="Email"
```

```
254         <input class="form-control mb-3" name="password" type="password"
placeholder="Password" required>
```

```
<button class="btn btn-brand w-100">Login</button>
```


257 <p class="text-center mt-3 mb-0">New here? Create
an account</p>

262 **return page(body)**


```
264 @app.route("/logout")
```

```
265 def logout():
```

266 `session.clear()`

```
267         flash("Logged out.")
```

```
268         return redirect(url_for("index"))
```



```
271 @app.route("/dashboard")
```


272 @login_required

```
273 def dashboard():
```

```
274         db = get_db()
```

```
275     user = current_user()
```

```
276         total_bookings = db.execute("SELECT COUNT(*) c FROM bookings WHERE user_id=? AND  
status='CONFIRMED'",
```

```
(user["id"],)).fetchone()["c"]
```

```
278         total_wishlist = db.execute("SELECT COUNT(*) c FROM wishlist WHERE user_id=?",
    (user["id"],)).fetchone()["c"]
```

```
279     total_packages = db.execute("SELECT COUNT(*) c FROM packages").fetchone()["c"]
```



```
280     body = ""
```

281 <h2 class="fw-bold mb-1">Hi {{ user['name'] }} 🙌</h2>

<p class="text-muted mb-4">What would you like to do today?</p>


```
285         <div class="col-md-4"><div class="stat-box text-center"><h3 class="fw-bold
mb-0">{{ total_packages }}</h3><small class="text-muted">Packages available</small></div></div>
```

```
286      <div class="col-md-4"><div class="stat-box text-center"><h3 class="fw-bold
mb-0">{{ total_bookings }}</h3><small class="text-muted">Your active bookings</small></div></div>
```

```
287         <div class="col-md-4"><div class="stat-box text-center"><h3 class="fw-bold
mb-0">{{ total_wishlist }}</h3><small class="text-muted">Saved in wishlist</small></div></div>
```


290 <div class="row g-3">

```
291         <div class="col-md-3 col-6"><a class="feature-btn" href="{{ url_for('packages') }}"><span
class="feature-emoji"><img alt="package icon" data-bbox="288 88 305 103"/></span>Browse Packages</a></div>
```

```
292         <div class="col-md-3 col-6"><a class="feature-btn"
href="{{ url_for('my_bookings') }}"><span class="feature-emoji">📅</span>My Bookings</a></div>
```

```
293         <div class="col-md-3 col-6"><a class="feature-btn" href="{{ url_for('wishlist') }}"><span
class="feature-emoji">❤️</span>Wishlist</a></div>
```

```
294         <div class="col-md-3 col-6"><a class="feature-btn" href="{ { url_for('profile') } }"><span
class="feature-emoji"><img alt="User profile icon" data-bbox="288 88 302 102"/></span>My Profile</a></div>
```



```
297         return page(body, user=user, total_bookings=total_bookings, total_wishlist=total_wishlist,  
total_packages=total_packages)
```



```
300 @app.route("/packages")
```

301 @login_required

```
302 def packages():
```

```
303         db = get_db()
```



```
304     user = current_user()
```

```
305     dest = request.args.get("destination", "").strip()
```

```
306     max_price = request.args.get("max_price", "").strip()
```

```
307         query = """SELECT p.*, i.slots_available FROM packages p JOIN inventory i ON  
i.package_id=p.id WHERE 1=1"""
```

```
308     params = []
```

```
309         if dest:
```

310

```
query += " AND p.destination LIKE ?"; params.append(f"%{dest}%")
```

```
311     if max_price:
```



```
312         query += " AND p.price <= ?"; params.append(max_price)
```

```
313     pkgs = db.execute(query, params).fetchall()
```

```
314         wished = {r["package_id"] for r in db.execute("SELECT package_id FROM wishlist WHERE  
user_id=?", (user["id"],))}
```

```
315     body = ""
```

```
<h3 class="mb-3"> Browse Travel Packages</h3>
```

317 <form class="row g-2 mb-4">

```
318         <div class="col-auto"><input class="form-control" name="destination"
placeholder="Destination" value="{{ request.args.get('destination','') }}"></div>
```

```
319         <div class="col-auto"><input class="form-control" name="max_price" placeholder="Max
price" value="{{ request.args.get('max_price', '') }}"></div>
```



```
<div class="col-auto"><button class="btn btn-brand">Search</button></div>
```



```
323      {% for p in pkgs %}
```



```
<span class="feature-emoji">{{ p['emoji'] }}</span>
```

```
<h5>{{ p['name'] }}</h5>
```



```
<p class="text-muted mb-1">{{ p['destination'] }} · {{ p['duration_days'] }} days</p>
```

```
<p class="small">{{ p['description'] }}</p>
```

```
330      <span class="price-tag mb-2" style="width:fit-content;">₹{{ '%.0f'|format(p['price'])  
}}</span>
```

```
331         <p class="small" {{ 'text-danger' if p['slots_available']==0 else 'text-  
success' }}">{{ p['slots_available'] }} slot(s) left</p>
```



```
333         <a class="btn btn-brand btn-sm flex-fill" href="{{ url_for('package_detail',  
pkg_id=p['id']) }}">View / Book</a>
```

```
<form method="post" action="{ { url_for('toggle_wishlist', pkg_id=p['id']) } }">
```

```
335         <button class="btn btn-sm {{ 'btn-danger' if p['id'] in wished else 'btn-outline-  
danger' }}">♥</button>
```



```
340      {% endfor %}
```



```
343     return page(body, pkgs=pkgs, request=request, wished=wished)
```



```
345 @app.route("/package/<int:pkg_id>")
```

346 @login_required

```
347 def package_detail(pkg_id):
```

```
348         db = get_db()
```

```
349     p = db.execute("""SELECT p.*, i.slots_available FROM packages p
```

```
350 JOIN inventory i ON i.package_id=p.id WHERE p.id=?""",  
(pkg_id,)).fetchone()
```

```
351         if not p:
```


352

```
return "Package not found", 404
```

```
353     body = ""
```

```
<div class="card p-4 fade-in" style="max-width:600px;">
```

```
<span class="feature-emoji">{{ p['emoji'] }}</span>
```


357

```
<h6 class="text-muted">{{ p['destination'] }} · {{ p['duration_days'] }} days</h6>
```

```
<p>{{ p['description'] }}</p>
```

```
359      <span class="price-tag mb-3" style="width:fit-content;">₹{{ '%.0f'|format(p['price']) }}  
</span>
```



```
<p>{{ p['slots_available'] }} slot(s) available</p>
```

```
361      {% if p['slots_available'] > 0 %}
```

```
<form method="post" action="{ { url_for('book_package', pkg_id=p['id']) } }">
```

```
<button class="btn btn-brand">Book Now</button>
```



```
365      {% else %}
```

```
<button class="btn btn-secondary" disabled>Sold Out</button>
```

```
367      {% endif %}
```



```
368         <a class="mt-3 d-inline-block" href="{{ url_for('packages') }}">⬅️ Back to all  
packages</a>
```



```
371     return page(body, p=p)
```



```
373 @app.route("/book/<int:pkg_id>", methods=["POST"])
```

374 @login_required

```
375 def book_package(pkg_id):
```



```
376         db = get_db()
```

```
377     user = current_user()
```

378 **try:**


```
380         row = db.execute("SELECT slots_available FROM inventory WHERE package_id=?",  
(pkg_id,)).fetchone()
```

```
381         if row is None or row["slots_available"] < 1:
```

```
db.execute("ROLLBACK")
```

```
flash("Sorry, this package is sold out.")
```



```
return redirect(url_for("package_detail", pkg_id=pkg_id))
```

```
385         db.execute("UPDATE inventory SET slots_available = slots_available - 1, version =  
version + 1 WHERE package_id=?", (pkg_id,))
```

```
386         db.execute("INSERT INTO bookings (user_id, package_id, status, created_at) VALUES  
(?,?,?,?)",
```

387

```
(user["id"], pkg_id, "CONFIRMED", datetime.utcnow().isoformat()))
```


389

```
flash("🎉 Booking confirmed! Check 'My Bookings'.")
```

```
390     except Exception as e:
```



```
flash(f"Booking failed: {e}")
```

```
393     return redirect(url_for("my_bookings"))
```



```
396 @app.route("/my-bookings")
```

397 @login_required

```
398 def my_bookings():
```

```
399         db = get_db()
```



```
400     user = current_user()
```

```
401     rows = db.execute("""SELECT b.*, p.name, p.destination, p.price, p.emoji FROM bookings b
```

```
JOIN packages p ON p.id=b.package_id
```

```
403                                     WHERE b.user_id=? ORDER BY b.created_at DESC""",
(user["id"],)).fetchall()
```

```
404      body = ""
```

405

```
<h3 class="mb-3"> My Bookings</h3>
```

```
406      {% if rows|length == 0 %}
```

```
407      <p class="text-muted">No bookings yet. <a href="{{ url_for('packages') }}">Browse  
packages &rarr;</a></p>
```



```
408      {% endif %}
```

```
409      {% for b in rows %}
```

410

```
<div class="card p-3 mb-2 d-flex flex-row align-items-center justify-content-between">
```



```
<span class="fs-4 me-2">{{ b['emoji'] }}</span>
```

```
<strong>{{ b['name'] }}</strong> - {{ b['destination'] }}
```

```
414         <span class="badge bg-{{ 'success' if b['status']=='CONFIRMED' else 'secondary' }}  
ms-2">{{ b['status'] }}</span>
```

```
415         <div class="small text-muted">Booked {{ b['created_at'][:10] }} · ₹{{ '%.0f'|
format(b['price']) }}</div>
```



```
417      {% if b['status'] == 'CONFIRMED' %}
```

418

```
<form method="post" action="{ { url_for('cancel_booking', booking_id=b['id']) } }">
```

```
<button class="btn btn-sm btn-outline-danger">Cancel</button>
```


421

```
{% endif %}
```



```
423      {% endfor %}
```



```
425         return page(body, rows=rows)
```



```
427 @app.route("/cancel/<int:booking_id>", methods=["POST"])
```

428 @login_required

```
429 def cancel_booking(booking_id):
```

```
430         db = get_db()
```

```
431     user = current_user()
```



```
432         b = db.execute("SELECT * FROM bookings WHERE id=? AND user_id=?", (booking_id,
user["id"])).fetchone()
```

```
433     if b and b["status"] == "CONFIRMED":
```

```
db.execute("BEGIN IMMEDIATE")
```

435

```
db.execute("UPDATE bookings SET status='CANCELLED' WHERE id=?", (booking_id,))
```

```
436         db.execute("UPDATE inventory SET slots_available = slots_available + 1 WHERE  
package_id=?", (b["package_id"],))
```

437

```
db.execute("COMMIT")
```

438

```
flash("Booking cancelled and slot released.")
```

```
439         return redirect(url_for("my_bookings"))
```


441 # ===== Feature: Wishlist

```
442 @app.route("/wishlist")
```

443 @login_required

```
444 def wishlist():
```

```
445         db = get_db()
```

```
446     user = current_user()
```

```
447     rows = db.execute("""SELECT p.*, i.slots_available FROM wishlist w
```



```
JOIN packages p ON p.id=w.package_id
```

```
JOIN inventory i ON i.package_id=p.id
```

```
WHERE w.user_id=?", (user["id"],)).fetchall()
```

```
451     body = ""
```

452

```
<h3 class="mb-3">❤️ My Wishlist</h3>
```

```
453      {% if rows|length == 0 %}
```

```
454      <p class="text-muted">Nothing saved yet. <a href="{{ url_for('packages') }}">Browse  
packages &rarr;</a></p>
```

```
455      {% endif %}
```



```
457      {% for p in rows %}
```



```
<span class="feature-emoji">{{ p['emoji'] }}</span>
```

```
<h5>{{ p['name'] }}</h5>
```

```
<p class="text-muted mb-1">{{ p['destination'] }} · {{ p['duration_days'] }} days</p>
```

```
463      <span class="price-tag mb-2" style="width:fit-content;">₹{{ '%.0f'|format(p['price'])  
}}</span>
```



```
465         <a class="btn btn-brand btn-sm flex-fill" href="{{ url_for('package_detail',  
pkg_id=p['id']) }}">View / Book</a>
```

```
<form method="post" action="{ { url_for('toggle_wishlist', pkg_id=p['id']) } }">
```

```
<button class="btn btn-sm btn-danger">Remove</button>
```



```
472      {% endfor %}
```



```
475         return page(body, rows=rows)
```



```
477 @app.route("/wishlist/toggle/<int:pkg_id>", methods=["POST"])
```

478 @login_required

```
479 def toggle_wishlist(pkg_id):
```



```
480         db = get_db()
```

```
481     user = current_user()
```

```
482         exists = db.execute("SELECT 1 FROM wishlist WHERE user_id=? AND package_id=?", (user["id"],  
pkg_id)).fetchone()
```

```
483         if exists:
```

```
484 db.execute("DELETE FROM wishlist WHERE user_id=? AND package_id=?", (user["id"],  
pkg_id))
```

485

```
flash("Removed from wishlist.")
```

486 **else:**

```
487 db.execute("INSERT INTO wishlist(user_id, package_id) VALUES (?,?)", (user["id"],  
pkg_id))
```



```
flash("Added to wishlist ❤️")
```

489 `db.commit()`

```
490         return redirect(request.referrer or url_for("packages"))
```



```
493 @app.route("/profile")
```

494 @login_required

```
495 def profile():
```



```
496         db = get_db()
```

```
497     user = current_user()
```

```
498         total_bookings = db.execute("SELECT COUNT(*) c FROM bookings WHERE user_id=? AND  
status='CONFIRMED'",
```



```
500     total_spent = db.execute("""SELECT COALESCE(SUM(p.price),0) s FROM bookings b
```

```
JOIN packages p ON p.id=b.package_id
```

```
502                                     WHERE b.user_id=? AND b.status='CONFIRMED'""",  
(user["id"],)).fetchone()["s"]
```

```
503     body = ""
```



```
<h3 class="mb-3"><img alt="User profile icon" data-bbox="318 81 333 93"/> My Profile</h3>
```

505 <div class="card p-4" style="max-width:500px;">

```
<p><strong>Name:</strong> {{ user['name'] }}</p>
```

```
<p><strong>Email:</strong> {{ user['email'] }}</p>
```

```
<p><strong>Member since:</strong> {{ user['created_at'][:10] }}</p>
```



```
<p><strong>Active bookings:</strong> {{ total_bookings }}</p>
```

511

```
<p><strong>Total spent:</strong> ₹{{ '%.0f'|format(total_spent) }}</p>
```



```
514     return page(body, user=user, total_bookings=total_bookings, total_spent=total_spent)
```



```
517 if __name__ == "__main__":
```

518 `init_db()`

```
519     app.run(debug=True)
```


C. Installation / Setup Instructions

1. Install Python 3.8 or later.
2. Open a terminal in the project folder.
3. Run pip install flask.
4. Run python app.py (or py app.py).
5. Open <http://127.0.0.1:5000> in a browser.
6. To reset the data, stop the server and delete the file travel.db; it is recreated on the next start.

D. User Manual

1. **Create an account:** click Register, enter your name, email and password, and press Register.
2. **Log in:** enter your email and password on the Login page. The dashboard opens.
3. **Find a package:** open Packages, type a destination and/or a maximum price and click Search.
4. **Book:** click View / Book on a package and then Book Now. A confirmation message appears and the booking is listed under My Bookings.
5. **Cancel:** open My Bookings and click Cancel next to a confirmed booking. The slot is returned to the package.
6. **Wishlist:** click the heart button on a package to save it, or Remove on the Wishlist page.
7. **Profile:** open Profile to see your active bookings and total spent.
8. **Log out:** click Logout in the navigation bar.